

DATA STRUCTURES

Unit-2: Stacks and Queues

Comprehensive Study Notes

1. Stack

Definition

A **stack** is a linear data structure that follows the **LIFO (Last In First Out)** principle. In a stack, insertion and deletion occur only at one end called the **top**.

STACK DIAGRAM (LIFO)

```
+-----+
|  3  | <-- TOP (Last In)
+-----+
|  7  |
+-----+
|  1  |
+-----+
|  9  | <-- BOTTOM (First In)
+-----+
```

Basic Operations

Operation	Description	Time Complexity
push(x)	Inserts element x onto the top of the stack	O(1)
pop()	Removes and returns the top element	O(1)
peek() / top()	Returns the top element without removing it	O(1)
isEmpty()	Checks if the stack is empty	O(1)
isFull()	Checks if the stack is full (for array implementation)	O(1)
size()	Returns the number of elements in the stack	O(1)

2. Stack Implementation Using Array

Structure

```
#define MAX 100

typedef struct {
    int arr[MAX];
    int top;
} Stack;
```

Initialize Stack

```
void init(Stack *s) {
    s->top = -1;
}
```

isEmpty

```
int isEmpty(Stack *s) {
    return s->top == -1;
}
```

isFull

```
int isFull(Stack *s) {
    return s->top == MAX - 1;
}
```

Push Operation

```

void push(Stack *s, int value) {
    if (isFull(s)) {
        printf("Stack Overflow!\n");
        return;
    }
    s->top++;
    s->arr[s->top] = value;
    printf("%d pushed to stack\n", value);
}

```

PUSH OPERATION

Before push(30):

```

+-----+
|  20  | <-- top
+-----+
|  10  |
+-----+
|      |
+-----+

```

After push(30):

```

+-----+
|  30  | <-- top
+-----+
|  20  |
+-----+
|  10  |
+-----+

```

Pop Operation

```

int pop(Stack *s) {
    if (isEmpty(s)) {
        printf("Stack Underflow!\n");
        return -1;
    }
    int value = s->arr[s->top];
    s->top--;
    return value;
}

```

POP OPERATION

Before pop():

```

+-----+
|  30  | <-- top
+-----+
|  20  |
+-----+
|  10  |
+-----+

```

After pop():

```

+-----+
|  20  | <-- top
+-----+
|  10  |
+-----+
|      |
+-----+

```

Peek / Top Operation

```
int peek(Stack *s) {
    if (isEmpty(s)) {
        printf("Stack is empty!\n");
        return -1;
    }
    return s->arr[s->top];
}
```

Display Operation

```
void display(Stack *s) {
    if (isEmpty(s)) {
        printf("Stack is empty!\n");
        return;
    }
    printf("Stack: ");
    for (int i = s->top; i >= 0; i--) {
        printf("%d ", s->arr[i]);
    }
    printf("\n");
}
```

Complete Program

```
#include <stdio.h>
#define MAX 100

typedef struct {
    int arr[MAX];
    int top;
} Stack;

void init(Stack *s) { s->top = -1; }
int isEmpty(Stack *s) { return s->top == -1; }
int isFull(Stack *s) { return s->top == MAX - 1; }

void push(Stack *s, int value) {
```

```
    if (isFull(s)) { printf("Stack Overflow!\n"); return; }
    s->arr[++s->top] = value;
    printf("%d pushed\n", value);
}

int pop(Stack *s) {
    if (isEmpty(s)) { printf("Stack Underflow!\n"); return -1; }
    return s->arr[s->top--];
}

int peek(Stack *s) {
    if (isEmpty(s)) { printf("Stack is empty!\n"); return -1; }
    return s->arr[s->top];
}

void display(Stack *s) {
    if (isEmpty(s)) { printf("Stack is empty!\n"); return; }
    for (int i = s->top; i >= 0; i--)
        printf("%d ", s->arr[i]);
    printf("\n");
}

int main() {
    Stack s;
    init(&s);
    push(&s, 10);
    push(&s, 20);
    push(&s, 30);
    display(&s);
    printf("Popped: %d\n", pop(&s));
    display(&s);
    printf("Top: %d\n", peek(&s));
    return 0;
}
```

Output:

```
10 pushed
20 pushed
30 pushed
Stack: 30 20 10
Popped: 30
Stack: 20 10
Top: 20
```

3. Stack Applications

3.1 Infix to Postfix Conversion

Infix: Operator between operands (e.g., $A + B$)

Postfix: Operator after operands (e.g., $AB+$)

Algorithm

1. Scan the infix expression from left to right
2. If operand, add to output
3. If '(', push to stack
4. If ')', pop until '(' and add popped to output
5. If operator, pop operators with higher/equal precedence, then push current operator
6. After scan, pop all remaining operators

Precedence Table

Operator	Symbol	Precedence	Associativity
Parentheses	()	Highest	-
Exponentiation	$\wedge$	3	Right to Left
Multiply / Divide	$*, /$	2	Left to Right
Add / Subtract	$+, -$	1	Left to Right

Example

Infix: $A + B * C$

Postfix: $A B C * +$

Program

```
#include <stdio.h>
#include <ctype.h>
#define MAX 100

char stack[MAX];
int top = -1;

void push(char c) {
    if (top < MAX - 1) stack[++top] = c;
}

char pop() {
    if (top == -1) return -1;
    return stack[top--];
}

int precedence(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}

void infixToPostfix(char infix[]) {
    char c;
    for (int i = 0; infix[i] != '\0'; i++) {
        c = infix[i];
        if (isalnum(c)) {
            printf("%c", c);
        } else if (c == '(') {
            push(c);
        } else if (c == ')') {
            while (top != -1 && stack[top] != '(')
                printf("%c", pop());
            pop(); // remove '('
        } else {
            while (top != -1 && precedence(stack[top]) >= precedence(c))
                printf("%c", pop());
            push(c);
        }
    }
    while (top != -1)
        printf("%c", pop());
}
```



```
int main() {  
    char expr[] = "A+B*C";  
    printf("Infix: %s\nPostfix: ", expr);  
    infixToPostfix(expr);  
    printf("\n");  
    return 0;  
}
```

Output: Infix: A+B*C Postfix: ABC*+

3.2 Expression Evaluation (Postfix)

Algorithm

1. Scan postfix expression from left to right
2. If operand, push onto stack
3. If operator, pop two operands, apply operator, push result back
4. At end, stack contains the final result

Program

```
#include <stdio.h>  
#include <ctype.h>  
#define MAX 100  
  
int stack[MAX];  
int top = -1;  
  
void push(int v) { stack[++top] = v; }  
int pop() { return stack[top--]; }  
  
int evaluatePostfix(char expr[]) {  
    int a, b;  
    for (int i = 0; expr[i] != '\0'; i++) {  
        if (isdigit(expr[i])) {  
            push(expr[i] - '0');  
        } else {  
            b = pop();  
            a = pop();
```

```
        switch (expr[i]) {
            case '+': push(a + b); break;
            case '-': push(a - b); break;
            case '*': push(a * b); break;
            case '/': push(a / b); break;
        }
    }
}
return pop();
}

int main() {
    char expr[] = "23*5+";
    printf("Postfix: %s\nResult: %d\n", expr, evaluatePostfix(expr));
    return 0;
}
```

Output: Postfix: 23*5+ Result: 11

3.3 Recursion

Recursion is a technique where a function calls itself. The system uses a stack to store the state of each recursive call.

Example: Factorial

```
#include <stdio.h>

int factorial(int n) {
    if (n <= 1)
        return 1;
    return n * factorial(n - 1);
}

int main() {
    printf("Factorial of 5: %d\n", factorial(5));
    return 0;
}
```

RECURSIVE CALL STACK FOR factorial(3)

```
factorial(3)
|-- calls factorial(2)
|   |-- calls factorial(1)
|   |   |-- returns 1
|   |-- returns 2 * 1 = 2
|-- returns 3 * 2 = 6
```

3.4 Balanced Parentheses

Algorithm

1. Scan expression from left to right
2. If opening bracket '({[' push onto stack
3. If closing bracket ')}]', pop from stack and check match
4. If mismatch or stack empty after scan, expression is unbalanced

Program

```
#include <stdio.h>
#define MAX 100

char stack[MAX];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }

int isMatchingPair(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

int areBalanced(char expr[]) {
    for (int i = 0; expr[i] != '\0'; i++) {
        if (expr[i] == '(' || expr[i] == '{' || expr[i] == '[') {
            push(expr[i]);
        } else if (expr[i] == ')' || expr[i] == '}' || expr[i] == ']') {
            if (top == -1 || !isMatchingPair(pop(), expr[i]))
                return 0;
        }
    }
}
```

```
        return top == -1;
    }

    int main() {
        char expr1[] = "({a+b}*{c-d})";
        char expr2[] = "({a+b}*{c-d})}";

        printf("%s: %s\n", expr1, areBalanced(expr1) ? "Balanced" : "Unbalanced");
        printf("%s: %s\n", expr2, areBalanced(expr2) ? "Balanced" : "Unbalanced");
        return 0;
    }
```

Output:

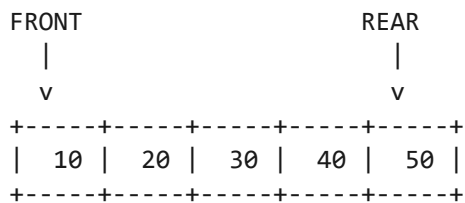
```
({a+b}*{c-d}): Balanced
({a+b}*{c-d})}: Unbalanced
```

4. Queue

Definition

A **queue** is a linear data structure that follows the **FIFO (First In First Out)** principle. Elements are inserted at the **rear** and removed from the **front**.

QUEUE DIAGRAM (FIFO)



Insert (enqueue) ---> at REAR
Delete (dequeue) ---> from FRONT

Basic Operations

Operation	Description	Time Complexity
enqueue(x)	Inserts element x at the rear of the queue	O(1)
dequeue()	Removes and returns the front element	O(1)
front()	Returns the front element without removing it	O(1)
isEmpty()	Checks if the queue is empty	O(1)
isFull()	Checks if the queue is full	O(1)

5. Queue Implementation Using Array

Structure

```
#define MAX 100

typedef struct {
    int arr[MAX];
    int front, rear;
} Queue;
```

Initialize Queue

```
void init(Queue *q) {
    q->front = -1;
    q->rear = -1;
}

int isEmpty(Queue *q) {
    return q->front == -1;
}

int isFull(Queue *q) {
    return q->rear == MAX - 1;
}
```

Enqueue Operation

```
void enqueue(Queue *q, int value) {
    if (isFull(q)) {
        printf("Queue Overflow!\n");
        return;
    }
    if (isEmpty(q))
        q->front = 0;
    q->arr[++q->rear] = value;
```

```
    printf("%d\n", value);
}
```

ENQUEUE OPERATION

Before enqueue(40):

FRONT=0 REAR=2

```

  |           |
  v           v
+---+---+---+---+
| 10 | 20 | 30 |   |
+---+---+---+---+
```

After enqueue(40):

FRONT=0 REAR=3

```

  |           |
  v           v
+---+---+---+---+
| 10 | 20 | 30 | 40 |
+---+---+---+---+
```

Deque Operation

```
int dequeue(Queue *q) {
    if (isEmpty(q)) {
        printf("Queue Underflow!\n");
        return -1;
    }
    int value = q->arr[q->front];
    if (q->front == q->rear) {
        q->front = -1;
        q->rear = -1;
    } else {
        q->front++;
    }
    return value;
}
```

DEQUEUE OPERATION

Before dequeue():

FRONT=0 REAR=3

```

  |           |
  v           v
+---+---+---+---+
| 10 | 20 | 30 | 40 |
+---+---+---+---+
```

After dequeue():

FRONT=1 REAR=3

```

  |           |
  v           v
+---+---+---+---+
|   | 20 | 30 | 40 |
+---+---+---+---+
```

Front Operation

```
int front(Queue *q) {
    if (isEmpty(q)) {
        printf("Queue is empty!\n");
        return -1;
    }
    return q->arr[q->front];
}
```

Display Operation

```
void display(Queue *q) {
    if (isEmpty(q)) {
        printf("Queue is empty!\n");
        return;
    }
    printf("Queue: ");
    for (int i = q->front; i <= q->rear; i++) {
        printf("%d ", q->arr[i]);
    }
    printf("\n");
}
```

Complete Program

```
#include <stdio.h>
#define MAX 100

typedef struct {
    int arr[MAX];
    int front, rear;
} Queue;

void init(Queue *q) { q->front = q->rear = -1; }
int isEmpty(Queue *q) { return q->front == -1; }
int isFull(Queue *q) { return q->rear == MAX - 1; }

void enqueue(Queue *q, int value) {
    if (isFull(q)) { printf("Queue Overflow!\n"); return; }
    if (isEmpty(q)) q->front = 0;
    q->arr[++q->rear] = value;
}
```



```
        printf("%d enqueued\n", value);
    }

    int dequeue(Queue *q) {
        if (isEmpty(q)) { printf("Queue Underflow!\n"); return -1; }
        int v = q->arr[q->front];
        if (q->front == q->rear) q->front = q->rear = -1;
        else q->front++;
        return v;
    }

    void display(Queue *q) {
        if (isEmpty(q)) { printf("Queue is empty!\n"); return; }
        for (int i = q->front; i <= q->rear; i++)
            printf("%d ", q->arr[i]);
        printf("\n");
    }

    int main() {
        Queue q;
        init(&q);
        enqueue(&q, 10);
        enqueue(&q, 20);
        enqueue(&q, 30);
        display(&q);
        printf("Dequeued: %d\n", dequeue(&q));
        display(&q);
        return 0;
    }
```

Output:

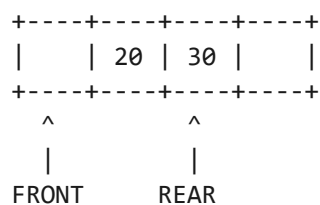
```
10 enqueued
20 enqueued
30 enqueued
Queue: 10 20 30
Dequeued: 10
Queue: 20 30
```

6. Circular Queue

Concept

A **circular queue** overcomes the limitation of a linear queue where space at the front cannot be reused. In a circular queue, the rear wraps around to the front when it reaches the end of the array, forming a circular structure.

CIRCULAR QUEUE



When rear reaches MAX-1, it wraps to index 0 if there is space available.

Advantages

- Better memory utilization – space at the front can be reused
- No need to shift elements
- Constant time $O(1)$ for both enqueue and dequeue

Implementation

```
#include <stdio.h>
#define MAX 5

typedef struct {
    int arr[MAX];
    int front, rear;
} CircularQueue;

void init(CircularQueue *q) {
    q->front = -1;
```

```
    q->rear = -1;
}

int isEmpty(CircularQueue *q) {
    return q->front == -1;
}

int isFull(CircularQueue *q) {
    return (q->rear + 1) % MAX == q->front;
}

void enqueue(CircularQueue *q, int value) {
    if (isFull(q)) {
        printf("Queue Overflow!\n");
        return;
    }
    if (isEmpty(q)) {
        q->front = 0;
        q->rear = 0;
    } else {
        q->rear = (q->rear + 1) % MAX;
    }
    q->arr[q->rear] = value;
    printf("%d enqueued\n", value);
}

int dequeue(CircularQueue *q) {
    if (isEmpty(q)) {
        printf("Queue Underflow!\n");
        return -1;
    }
    int value = q->arr[q->front];
    if (q->front == q->rear) {
        q->front = -1;
        q->rear = -1;
    } else {
        q->front = (q->front + 1) % MAX;
    }
    return value;
}

void display(CircularQueue *q) {
    if (isEmpty(q)) {
        printf("Queue is empty!\n");
        return;
    }
}
```

```
    }
    printf("Circular Queue: ");
    int i = q->front;
    while (1) {
        printf("%d ", q->arr[i]);
        if (i == q->rear) break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}

int main() {
    CircularQueue q;
    init(&q);
    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);
    display(&q);
    printf("Dequeued: %d\n", dequeue(&q));
    enqueue(&q, 40);
    enqueue(&q, 50);
    display(&q);
    return 0;
}
```

Output:

```
10 enqueued
20 enqueued
30 enqueued
Circular Queue: 10 20 30
Dequeued: 10
40 enqueued
50 enqueued
Circular Queue: 20 30 40 50
```

7. Deque (Double-Ended Queue)

Definition

A **Deque** (Double-Ended Queue) is a linear data structure in which insertion and deletion can be performed at both ends (front and rear).

DEQUE (Double-Ended Queue)

Insert/Delete
at FRONT

|
v

```
+-----+-----+-----+-----+
|  10  |  20  |  30  |  40  |
+-----+-----+-----+-----+
```

Insert/Delete
at REAR

|
v

Types of Deque

Type	Description
Input-Restricted Deque	Insertion allowed only at one end, deletion at both ends
Output-Restricted Deque	Deletion allowed only at one end, insertion at both ends

Operations

Operation	Description
insertFront(x)	Insert element at the front
insertRear(x)	Insert element at the rear
deleteFront()	Delete element from the front
deleteRear()	Delete element from the rear

getFront()	Get the front element
getRear()	Get the rear element

8. Priority Queue

Definition

A **priority queue** is a special type of queue in which each element is assigned a priority. Elements with higher priority are dequeued before elements with lower priority, regardless of their order of arrival.

Types

Type	Description
Ascending Priority Queue	Smallest element has highest priority (dequeued first)
Descending Priority Queue	Largest element has highest priority (dequeued first)

Implementation Approaches

Approach	Enqueue	Dequeue
Using Array (unsorted)	$O(1)$	$O(n)$ – search for highest priority
Using Array (sorted)	$O(n)$ – insert in sorted order	$O(1)$
Using Heap	$O(\log n)$	$O(\log n)$

Applications of Priority Queue

- CPU Scheduling (process priorities)
- Dijkstra's shortest path algorithm
- Data compression (Huffman coding)
- Event-driven simulation

- Operating system task management

9. Comparison: Stack vs Queue

Feature	Stack	Queue
Principle	LIFO (Last In First Out)	FIFO (First In First Out)
Insertion	Called push, at top	Called enqueue, at rear
Deletion	Called pop, from top	Called dequeue, from front
Ends Used	One end (top)	Two ends (front and rear)
Applications	Expression evaluation, recursion, undo	Scheduling, buffering, BFS
Array Implementation	Single pointer (top)	Two pointers (front, rear)

10. Quick Revision

Stack Summary

STACK

Principle: LIFO

Operations: push, pop, peek, isEmpty, isFull

Applications: infix-to-postfix, expression evaluation,
recursion, balanced parentheses, undo

Implementations: Array, Linked List

Queue Types Summary

QUEUES

+- Linear Queue (FIFO)

+- Circular Queue (wraps around)

+- Deque (insert/delete at both ends)

+- Priority Queue (based on priority)

Time Complexity Summary

Data Structure	Insertion	Deletion	Peek/Front
Stack (Array)	O(1)	O(1)	O(1)
Queue (Array)	O(1)	O(1)	O(1)
Circular Queue	O(1)	O(1)	O(1)

11. Questions

Short Answer Questions (2-5 Marks)

1. What is a stack? Explain the LIFO principle.
2. What are the basic operations on a stack?
3. What is stack overflow and stack underflow?
4. What is a queue? Explain the FIFO principle.
5. What are the basic operations on a queue?
6. What is a circular queue? What are its advantages?
7. What is a Deque? What are its types?
8. What is a priority queue?
9. What is the difference between a stack and a queue?
10. What are the applications of stacks?
11. What are the applications of queues?

Long Answer Questions (10 Marks)

1. Explain stack implementation using arrays with all operations (push, pop, peek, isFull, isEmpty, display).
2. Explain the algorithm and program for infix to postfix conversion using stack.
3. Explain postfix expression evaluation using stack with an example.
4. Explain balanced parentheses checking using stack.
5. Explain queue implementation using arrays with all operations.
6. Explain circular queue with its advantages and implementation.
7. Explain Deque and priority queue with examples.
8. Compare linear queue and circular queue.

Objective Questions (1 Mark)

1. Stack follows:
(a) FIFO (b) LIFO (c) Both (d) None

Answer: (b)

2. Queue follows:

- (a) FIFO (b) LIFO (c) Both (d) None

Answer: (a)

3. Time complexity of push operation in stack is:

- (a) $O(1)$ (b) $O(n)$ (c) $O(n^2)$ (d) $O(\log n)$

Answer: (a)

4. In a stack, insertion and deletion occur at:

- (a) Top (b) Bottom (c) Middle (d) Anywhere

Answer: (a)

5. In a queue, insertion occurs at:

- (a) Front (b) Rear (c) Both ends (d) Middle

Answer: (b)

6. Which queue reuses empty spaces at the front?

- (a) Linear Queue (b) Circular Queue (c) Deque (d) Priority Queue

Answer: (b)

7. A Deque stands for:

- (a) Double-Ended Queue (b) Dual-Ended Queue (c) Double-Entry Queue (d) None

Answer: (a)

8. The top of an empty stack is:

- (a) 0 (b) -1 (c) 1 (d) MAX

Answer: (b)

End of Unit-2 Notes — Data Structures (HPU BCA Sem-2)